FATEC POMPEIA SHUNJI NISHIMURA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório Consolidado

Arquitetura do SaaS e Atividades do 2º Bimestre

Avaliação Individual · 2026.2

Cleiton Rodrigues de Souza

Produto: **Conta Comigo** — Controle Financeiro Colaborativo
https://github.com/cleitongrodrigues/Controle_Financeiro

Prof. Allan Lincoln Rodrigues Siriani
Pompeia - SP, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura do SaaS 3
3.1 Visão geral e componentes 3
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados do domínio 6
3.6 Segurança 7
3.7 Observabilidade, CI/CD e documentação da API 7
3.8 Camadas IaaS, PaaS e SaaS do projeto 7
4 Quadro de entregas 8
5 Atividades realizadas 8
5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe 8
5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve 9
5.3 Atividade 9 — Fork e plano de negócio 9
5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes 9
5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe 12
6 Considerações finais 13
7 Declaração de autoria 13

1 Introdução

A disciplina de **Introdução à Computação em Nuvem (ISW055)**, ministrada pelo Prof. Allan Lincoln Rodrigues Siriani no curso de Sistemas Inteligentes da Fatec Pompeia, tem como objetivo capacitar os estudantes no projeto, desenvolvimento, conteinerização, entrega contínua e gerenciamento de aplicações distribuídas em ambientes de nuvem. Mais do que construir código isolado, a proposta pedagógica desafia os alunos a vivenciarem o ciclo completo da engenharia de software moderna: microsserviços, isolamento de recursos, computação elástica, mensageria assíncrona, gateways de pagamento e o modelo de negócios de Software como Serviço (SaaS).

Ao longo do primeiro bimestre, a base de estudo foi estruturada sobre um catálogo de filmes de demonstração. Embora funcional para o aprendizado inicial de Docker, Portainer, autenticação JWT e banco relacional, tratava-se de um sistema acadêmico genérico. No segundo bimestre, o projeto deu um salto significativo de maturidade: o antigo catálogo do Tom Hanks foi totalmente aposentado e deu lugar ao **Conta Comigo**, um produto SaaS de controle financeiro projetado para atender demandas reais de mercado, com regras de negócio autênticas, multi-tenancy robusto, auditoria e monetização recorrente.

O **Conta Comigo** resolve uma das maiores dores da vida financeira cotidiana: a divisão caótica e desorganizada de contas e despesas entre casais, membros da mesma família ou pessoas que dividem moradia (repúblicas). Tradicionalmente, esse controle é feito por planilhas manuais propensas a erro, fotos de notas fiscais espalhadas em conversas de WhatsApp ou discussões sobre "quem pagou o quê". A plataforma unifica as contas individuais do usuário e permite criar contas conjuntas compartilhadas por meio de códigos de convite, com anexação de comprovantes de pagamento via armazenamento em nuvem S3 (MinIO), consolidação automática de saldos e controle rigoroso de acessos por papéis.

Este relatório consolidado (P2) documenta integralmente a arquitetura do sistema em produção no subdomínio `https://cleiton-souza-isw055.lapps.studio`, descrevendo os microsserviços componentes, o fluxo de requisições, o ciclo de vida das assinaturas recorrentes com Stripe, o modelo relacional de dados, os padrões de segurança e as camadas IaaS, PaaS e SaaS aplicadas. Adicionalmente, apresenta o quadro comprobatório das atividades 7 a 11 com suas respectivas evidências, datas e análises técnicas.

2 Metodologia

O desenvolvimento das entregas do segundo bimestre seguiu uma metodologia incremental, fundamentada em boas práticas de versionamento no Git, conteinerização com Docker Compose e implantação orientada a microsserviços no Portainer.

As atividades 7 (cobrança com Stripe Checkout) e 8 (documentação das camadas de nuvem) não foram entregues no prazo e constam como tal no quadro da seção 4. O produto foi desenvolvido a partir da atividade 9, no repositório público **Controle_Financeiro** (`github.com/cleitongrodrigues/Controle_Financeiro`), que contém o plano de negócio e o código das atividades 10 e 11.

A partir desse ponto, as atividades 10 (construção do domínio de controle financeiro, tabelas no banco, telas, upload no MinIO e RBAC) e 11 (assinaturas recorrentes, limites na API, webhooks idempotentes e painel administrativo com MRR) foram executadas diretamente no repositório novo. Em conformidade com as diretrizes da disciplina, a infraestrutura de hospedagem foi mantida sem ruptura: a mesma máquina virtual (VM Linux na nuvem da Fatec), o mesmo subdomínio público e a mesma stack do Portainer continuaram operando os novos containers sob o limite de 1 GB de memória RAM por serviço.

Todas as datas e horários de conclusão reportados neste documento foram extraídos fidedignamente do histórico criptográfico do Git (`git log --date=format:"%d/%m/%Y %H:%M" --format="%h | %ad | %s"`) (hash e data de cada commit), conferidas na página do commit no GitHub. As evidências visuais comprovam a conformidade das telas, endpoints, respostas HTTP e registros de logs com o sistema em execução em ambiente real.

3 Arquitetura do SaaS

A arquitetura do **Conta Comigo** é orientada a microsserviços desacoplados e distribuídos em containers Docker, operando sob uma rede interna dedicada (`conta-comigo-network`) e expondo apenas as interfaces públicas indispensáveis por meio de um proxy reverso com certificado SSL gerenciado pela infraestrutura da nuvem.

3.1 Visão geral e componentes

O ecossistema é formado por serviços especializados que interagem de forma síncrona (REST/HTTP) para requisições de negócio e autenticação, e assíncrona (Streams) para auditoria e logs:

Componente	Responsabilidade	Tecnologia / Imagem	Exposição
Frontend (App)	Interface Web responsiva (SPA), landing page pública institucional e dashboard financeiro.	React 18, Vite, Nginx (Alpine)	Público (Porta 8201 / Domínio)
Backend Core	API REST central, regras de negócio financeiras, controle multi-tenancy, validação de limites de plano e upload.	Node.js 22, Express.js	Público (Porta 3000 / Domínio)
Auth Service	Microsserviço de autenticação, geração e validação de tokens JWT, controle de papéis (RBAC) e recuperação de senha.	Node.js 22, Express, Bcrypt, JWT	Interno (Porta 3001)
Log Service	Microsserviço de auditoria e telemetria, recepção de eventos de negócio e consulta para administradores.	Node.js 22, Redis Client	Interno (Porta 3002)
Redis	Armazenamento em memória e mensageria de alta performance para a trilha de auditoria via Redis Streams.	Redis 7 (Alpine)	Interno (Porta 6379)
MySQL / MariaDB	Banco de dados relacional persistente com as tabelas de usuários, contas conjuntas, transações e idempotência.	MySQL (servidor externo à stack)	Externo, acessado via DB_HOST (porta 3306)
MinIO S3	Servidor de Object Storage compatível com a API AWS S3 para arquivamento seguro de notas fiscais e comprovantes.	MinIO (Elestio Latest)	Interno (Porta 9000)
Stripe (SaaS Externo)	Gateway de pagamentos para assinaturas recorrentes, cobrança via cartão de crédito, webhooks e portal do cliente.	Stripe API (v3 / Hosted Checkout)	Externo (Webhook ➔ Backend)

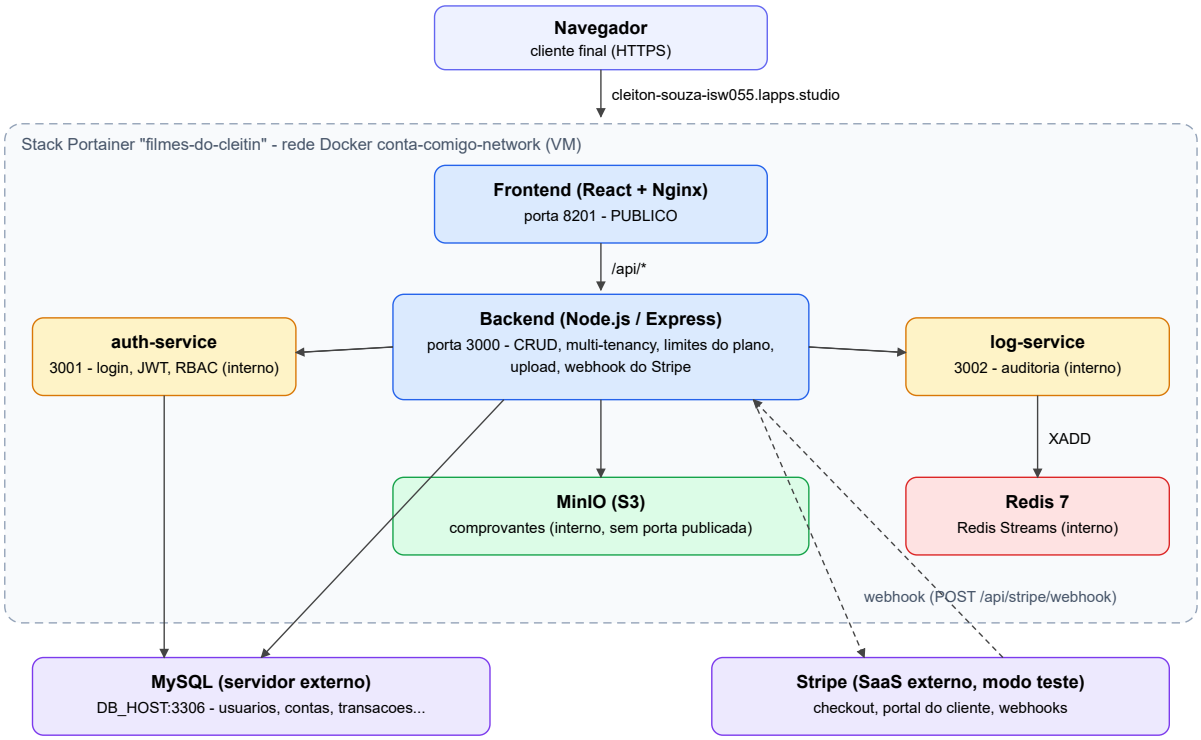


Figura 1: Diagrama de componentes: o que é público e o que existe só na rede interna do Docker

3.2 Onde roda

O sistema opera na mesma stack do Portainer utilizada desde o início do semestre letivo:

- **Nome da Stack no Portainer:** `controle-financeiro-cleiton` (stack do tipo Compose, criada em 01/11/2026 às 23:03).
- **Repositório Fonte:** `https://github.com/cleitongrodrigues/Controle_Financeiro` (branch `main`).
- **Ambiente de Hospedagem:** máquina virtual fornecida pela disciplina, administrada por meio do Portainer.
- **Limites de Recursos:** a disciplina estabelece 1 GB de memória RAM por container. Esse limite não está declarado no `docker-compose.yml` do repositório, então depende da configuração da stack no Portainer.
- **URL Pública de Produção:** `https://cleiton-souza-isw055.lapps.studio`
- **Variáveis de Ambiente da Stack:** Todas as configurações sensíveis estão isoladas no painel do Portainer e nunca expostas no código público. As variáveis configuradas incluem: `PORT`, `AUTH_SERVICE_PORT`, `AUTH_SERVICE_URL`, `LOG_SERVICE_URL`, `DB_HOST`, `DB_PORT`, `DB_USER`, `DB_PASSWORD`, `DB_NAME`, `DB_DIALECT`, `JWT_SECRET`, `MAILTRAP_HOST`, `MAILTRAP_PORT`, `MAILTRAP_USER`, `MAILTRAP_PASS`, `MAILTRAP_FROM`, `MINIO_ENDPOINT`, `MINIO_PUBLIC_URL`, `MINIO_ACCESS_KEY`, `MINIO_SECRET_KEY`, `MINIO_BUCKET_NAME`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET` e `STRIPE_PRICE_ID_PRO`.

3.3 Fluxo de uma requisição

Para ilustrar o percurso completo de uma chamada no sistema, toma-se como exemplo o caso de uso: **Membro ou Titular cria uma nova transação financeira com upload de comprovante fiscal**:

1. **Início no Navegador:** O usuário preenche o formulário da transação no Dashboard (título, valor, categoria, tipo e arquivo PDF/imagem) e clica em "Salvar". O frontend envia uma requisição `POST /api/transacoes` no formato `multipart/form-data` com o header `Authorization: Bearer <token_jwt>`.
2. **Roteamento e Proxy:** O tráfego HTTPS chega ao domínio `cleiton-souza-isw055.lapps.studio`, que responde por trás da Cloudflare, e é encaminhado até os containers da stack.
3. **Autenticação e Interceptação:** O middleware `auth.js` do backend valida a assinatura e a validade temporal do JWT por meio do `JWT_SECRET`. Se válido, injeta os dados decodificados do usuário (`id`, `nome`, `email`, `role`) em `req.user`.

4. **Validação Multi-Tenancy e RBAC:** O `transactionsController` consulta a tabela `membros_conta`. Ele verifica se o usuário autenticado de fato pertence à conta financeira indicada no corpo da requisição. Se não pertencer, a requisição é imediatamente abortada com código HTTP 403 Forbidden.
5. **Validação de Quota do Plano (Atividade 11):** O backend consulta o plano ativo do usuário na tabela `usuarios`. Se o plano for `free`, conta quantas transações o usuário já criou no mês corrente. Caso o total seja maior ou igual ao limite de 3 transações, a API rejeita o registro com HTTP 403 Forbidden e orienta o upgrade para o Plano Pro.
6. **Armazenamento de Objeto no MinIO:** Estando a requisição autorizada, o arquivo em memória é despachado via stream pelo `storageService.js` para o bucket `conta-comigo-comprovantes` no container `minio:9000`, gerando um caminho identificador seguro.
7. **Persistência Relacional no Banco:** A transação é inserida com instrução SQL `INSERT INTO transacoes` no MySQL, vinculando as chaves estrangeiras da conta, do usuário, da categoria e a URL do comprovante no MinIO.
8. **Disparo Assíncrono de Auditoria:** O backend faz uma requisição HTTP interna para o `log-service:3002`, que por sua vez grava o evento `transacao_criada` no comando `XADD logs:stream *` do container `redis:6379`.
9. **Resposta:** O backend retorna HTTP 201 Created com o payload JSON da transação, e a interface atualiza o saldo consolidado e o extrato instantaneamente.

3.4 Fluxo de assinatura

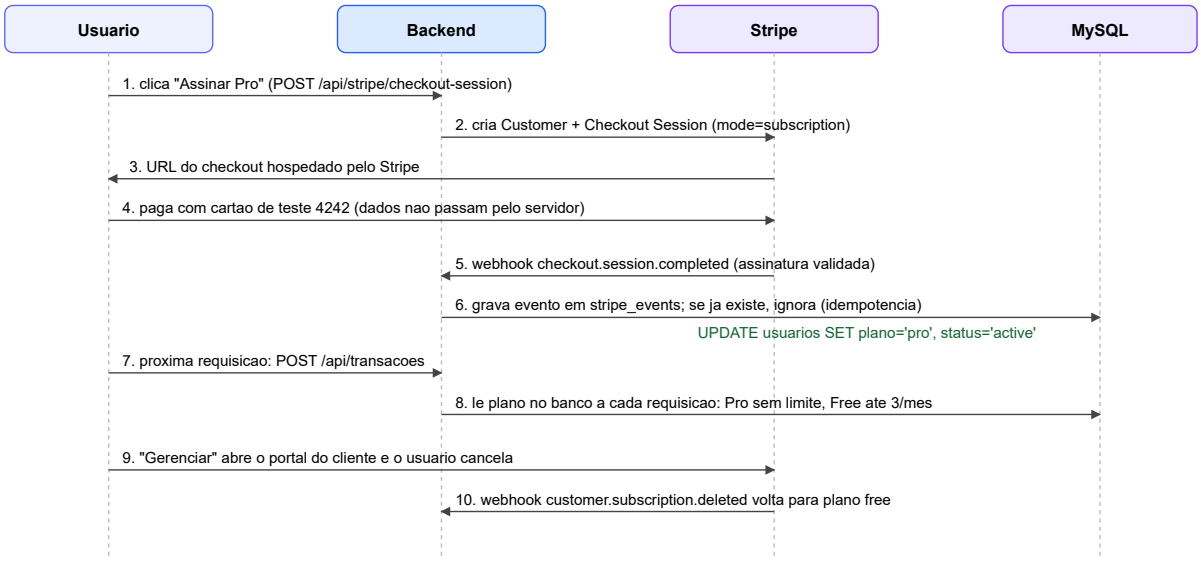


Figura 2: Fluxo de assinatura: checkout, webhook, plano no banco e cancelamento

O ciclo de vida da monetização do SaaS é suportado integralmente pelo Stripe em modo de teste:

1. **Criação da Sessão de Checkout:** Ao clicar em "Assinar Pro (R$ 19,90/mês)", o frontend chama `POST /api/stripe/checkout-session`. O backend busca ou registra o usuário como `Customer` no Stripe, associa o `metadata: { usuario_id }` e cria uma Checkout Session com `mode: 'subscription'` ligada ao `STRIPE_PRICE_ID_PRO`. O backend devolve a URL segura do checkout hospedado pelo Stripe.
2. **Pagamento com Cartão de Teste:** O cliente é direcionado à página do Stripe e informa os dados do cartão de teste institucional (`4242 4242 4242 4242`, validade futura, CVC `123`). O Stripe processa a assinatura e redireciona o navegador para a URL de sucesso no frontend.
3. **Processamento Seguro por Webhook:** O plano do usuário **nunca** é alterado pelo redirecionamento da tela. A mudança depende exclusivamente da notificação enviada pelo Stripe para `POST /api/stripe/webhook`:

- **Validação de Assinatura:** O backend captura o `rawBody` da requisição e valida a assinatura criptográfica através do cabeçalho `stripe-signature` e do `STRIPE_WEBHOOK_SECRET`. Se a assinatura for forjada ou inválida, a chamada é rejeitada com HTTP 400.
- **Idempotência Garantida:** Antes de processar, o backend verifica se o `event.id` já existe na tabela `stripe_events`. Se já existir (evento reenviado pelo Stripe), retorna HTTP 200 `{ duplicate: true }` sem executar comandos redundantes no banco.
- **Ativação Pro:** No evento `checkout.session.completed`, o backend executa `UPDATE usuarios SET plano = 'pro', subscription_status = 'active', stripe_customer_id = ?, stripe_subscription_id = ? WHERE id = ?`. O evento `assinatura_pro_iniciada` é registrado no log.

4. **Liberação Imediata na API:** Na próxima requisição do usuário, a API detecta `plano = 'pro'` e remove a trava de 3 transações por mês, liberando uso irrestrito.
5. **Portal do Cliente e Cancelamento:** O assinante tem acesso ao botão **"⭐ PRO (Gerenciar)"**, que chama `POST /api/stripe/portal-session` e abre o Stripe Customer Portal. Caso o cliente opte por cancelar a assinatura (configurada para cancelamento imediato em teste), o Stripe despacha o webhook `customer.subscription.deleted`, que rebaixa o registro para `plano = 'free'` e `subscription_status = 'canceled'`.
6. **Inadimplência:** Caso ocorra falha na renovação do cartão, o evento `invoice.payment_failed` altera o status para `past_due` e revoga imediatamente os privilégios Pro.

3.5 Modelo de dados do domínio

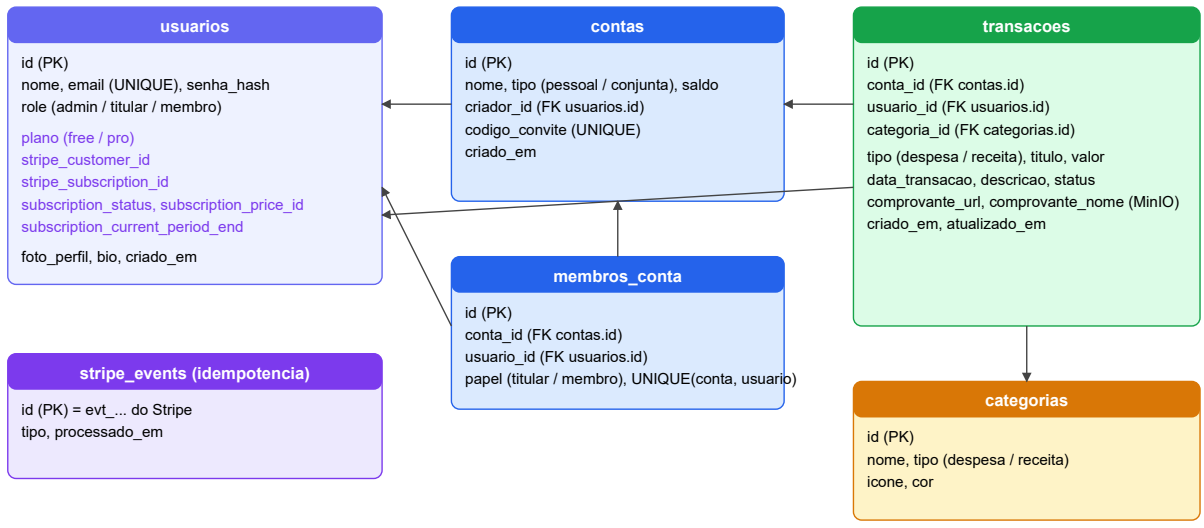


Figura 3: Modelo de dados do domínio e chaves estrangeiras

O modelo de dados relacional foi concebido sob a terceira forma normal (3NF), garantindo integridade referencial com chaves estrangeiras e índices otimizados para consultas multi-tenancy:

- **usuarios:** `id` (PK), `nome`, `email` (UNIQUE), `senha_hash`, `role` (admin/titular/membro), `plano` (free/pro), `stripe_customer_id`, `stripe_subscription_id`, `subscription_status`, `subscription_current_period_end`, `criado_em`.
- **contas:** `id` (PK), `nome`, `tipo` (pessoal/conjunta), `saldo`, `criador_id` (FK ➔ `usuarios.id`), `codigo_convite` (UNIQUE), `criado_em`.
- **membros_conta:** `id` (PK), `conta_id` (FK ➔ `contas.id`), `usuario_id` (FK ➔ `usuarios.id`), `papel` (titular/membro), `criado_em`. Garante a relação N:N entre usuários e contas conjuntas com a restrição `UNIQUE (conta_id, usuario_id)`.
- **categorias:** `id` (PK), `nome`, `tipo` (despesa/receita), `icone`, `cor`, `criado_em`.

- **transacoes:** `id` (PK), `conta_id` (FK ➔ `contas.id`), `usuario_id` (FK ➔ `usuarios.id`), `categoria_id` (FK ➔ `categorias.id`), `tipo` (despesa/receita), `titulo`, `valor`, `data_transacao`, `descricao`, `comprovante_url`, `comprovante_nome`, `status` (pago/pendente), `criado_em`.
- **stripe_events:** `id` (PK, string do Stripe `evt_...`), `tipo`, `processado_em`. Tabela dedicada a assegurar idempotência nos webhooks.

3.6 Segurança

A política de segurança da informação do SaaS atua em três camadas defensivas: controle de acesso baseado em papéis (RBAC), isolamento estrito de dados por usuário/conta (Multi-tenancy) e blindagem de segredos:

Ação de Negócio	Admin	Titular	Membro
Criar e gerenciar conta própria	Sim	Sim	Sim
Lançar receitas e despesas com anexo	Sim	Sim	Sim
Excluir transações de outros membros	Sim	Sim	Não (403)
Excluir conta financeira conjunta	Sim	Sim	Não (403)
Convidar membros para a conta	Sim	Sim	Não (403)
Acessar dados de contas alheias	Sim	Não (403)	Não (403)
Visualizar métricas e MRR (/logs)	Sim	Não (403)	Não (403)

Isolamento Multi-tenancy: O sistema impede qualquer acesso indevido por troca de parâmetros na URL (Insecure Direct Object References - IDOR). As rotas `/api/transacoes/:id` e `/api/contas/:id` invocam a função `checkUserAccountAccess`, que consulta o vínculo do usuário na tabela `membros_conta`. Se o solicitante não for membro daquela conta e não possuir privilégios globais de `admin`, o backend devolve status HTTP 403 Forbidden e dispara um log de tentativa de violação de segurança no Redis.

Gestão de Segredos: O repositório git não contém chaves de API, senhas ou tokens. O arquivo `.env.example` versionado lista apenas a nomenclatura das variáveis. Valores reais (chaves do Stripe, senhas de banco e credenciais do MinIO) residem unicamente nas variáveis de ambiente da stack no Portainer.

3.7 Observabilidade, CI/CD e documentação da API

- **Health Checks:** Todos os microsserviços do Docker Compose contam com diretivas de `healthcheck` ativas testando endpoints HTTP locais (`/health`) e utilitários (`redis-cli ping`), com inicialização encadeada (`depends_on` com condição `service_healthy`).
- **Trilha de Auditoria com Redis Streams:** Cada ação crítica do sistema (autenticação, criação de transação, upload de documento, bloqueios de segurança e eventos de pagamento do Stripe) gera um evento assíncrono publicado no Redis via comando `XADD`, permitindo rastreabilidade completa com carimbo de tempo, ID do usuário, ação e IP de origem.
- **Painel de Telemetria e MRR:** A rota `/api/admin/metrics` e a tela `/logs` disponibilizam ao administrador métricas consolidadas em tempo real: total de usuários divididos por plano (Free vs Pro), contagem de assinantes ativos e faturamento mensal recorrente estimado (MRR).

3.8 Camadas IaaS, PaaS e SaaS do projeto

Camada	O que representa no projeto	Responsabilidade de Gestão
IaaS	Máquina Virtual Linux provisionada com vCPUs, memória RAM dedicada, volumes de disco virtual, rede e roteamento de borda na nuvem da faculdade.	Equipe de Infraestrutura da

Camada	O que representa no projeto	Responsabilidade de Gestão
		Fatec Pompeia / Lapps Studio
PaaS	Motor Docker Engine, plataforma de orquestração Portainer, instâncias conteinerizadas do MariaDB/MySQL, Redis 7 e MinIO S3.	Aluno (configuração da stack) e Fatec (manutenção do runtime)
SaaS	1. O produto final entregue aos usuários: **Conta Comigo** (Web e API). 2. Os serviços em nuvem consumidos: Stripe (Billing), GitHub (repositório e versionamento) e Mailtrap (SMTP).	Aluno (desenvolvimento do Conta Comigo) e Provedores Externos (Stripe/GitHub)

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium — cobrança de verdade com Stripe	09/10/2026	—	não entregue
8	SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve	16/10/2026	—	não entregue
9	Fork e plano de negócio	23/10/2026	01/11/2026 21:50	entregue
10	Seu negócio no ar — o domínio no lugar dos filmes	06/11/2026	01/11/2026 22:11	entregue
11	SaaS completo — planos, limites e assinatura com Stripe	13/11/2026	11/11/2026 10:53	entregue

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium — cobrança de verdade com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	—
Situação	não entregue
Evidência	Nenhuma: não há commit com a integração do Stripe no histórico do repositório Filmes.
Link	https://github.com/cleitongrodrigues/Filmes

Situação. Esta atividade não foi entregue no prazo. O histórico do repositório do catálogo de filmes termina em 06/10/2026 (entrega da P1) e não contém nenhum commit de integração com o Stripe, então não há data de

entrega nem print que eu possa apresentar como prova. A integração com o Stripe só foi construída depois, na atividade 11, e aparece lá como assinatura recorrente.

5.2 Atividade 8 — SaaS, PaaS, IaaS — e a documentação que o projeto nunca teve

Descrição	Camadas de nuvem do próprio projeto e documentação reversa: especificação, arquitetura, regras de negócio e modelo de dados.
Data planejada	16/10/2026
Data realizada	—
Situação	não entregue
Evidência	Nenhuma: não existe DOCUMENTACAO.md nem commit da atividade 8 no repositório Filmes.
Link	https://github.com/cleitongrodrigues/Filmes

Situação. Esta atividade não foi entregue no prazo: não existe o arquivo `DOCUMENTACAO.md` nem commit correspondente. Como compensação parcial, a seção 3.8 deste relatório descreve as camadas IaaS, PaaS e SaaS do produto final, e o README do repositório novo traz o diagrama de arquitetura.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, modelo de receita, mapa de reaproveitamento) e backlog em issues.
Data planejada	23/10/2026
Data realizada	01/11/2026 21:50
Situação	entregue
Evidência	GitHub — commit a0212c3 (plano de negócio no README do repositório novo)
Link	https://github.com/cleitongrodrigues/Controle_Financeiro

O que foi feito. Realizou-se a transição estratégica do projeto acadêmico para um negócio real de Software como Serviço, criando o repositório público `Controle_Financeiro`. Redigiu-se o plano de negócio do **Conta Comigo** com estudo de mercado, persona (Mariana Lopes), Lean Canvas completo, mapa de reaproveitamento técnico (De/Para entre filmes e finanças).

Dificuldades e como foram resolvidas. O desafio foi desenhar um mapa de reaproveitamento de código que preservasse todos os microsserviços já estabilizados (auth-service, log-service, redis, minio) sem desperdiçar o esforço construído nas etapas anteriores, adaptando apenas o domínio semântico para contas e transações financeiras.

5.4 Atividade 10 — Seu negócio no ar — o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis do negócio, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	01/11/2026 22:11
Situação	entregue
Evidência	GitHub — commit 3bde1ea, URL no ar e prints de tela
Link	https://github.com/cleitongrodrigues/Controle_Financeiro/commit/3bde1ea

O que foi feito. O domínio financeiro foi completamente implementado em produção: criação das tabelas `contas`, `membros_conta`, `categorias` e `transacoes` no MySQL com scripts de migração e seed versionados; landing page pública responsiva para visitantes sem login; CRUD financeiro com anexo de comprovantes fiscais no MinIO; isolamento multi-tenancy rigoroso com resposta 403 Forbidden em tentativas de acesso indevido; e registro de eventos de negócio no Redis Streams. Todos os vestígios de filmes e da API TMDB foram erradicados.



Figura 4: Atividade 10 — Landing page pública institucional do Conta Comigo no ar

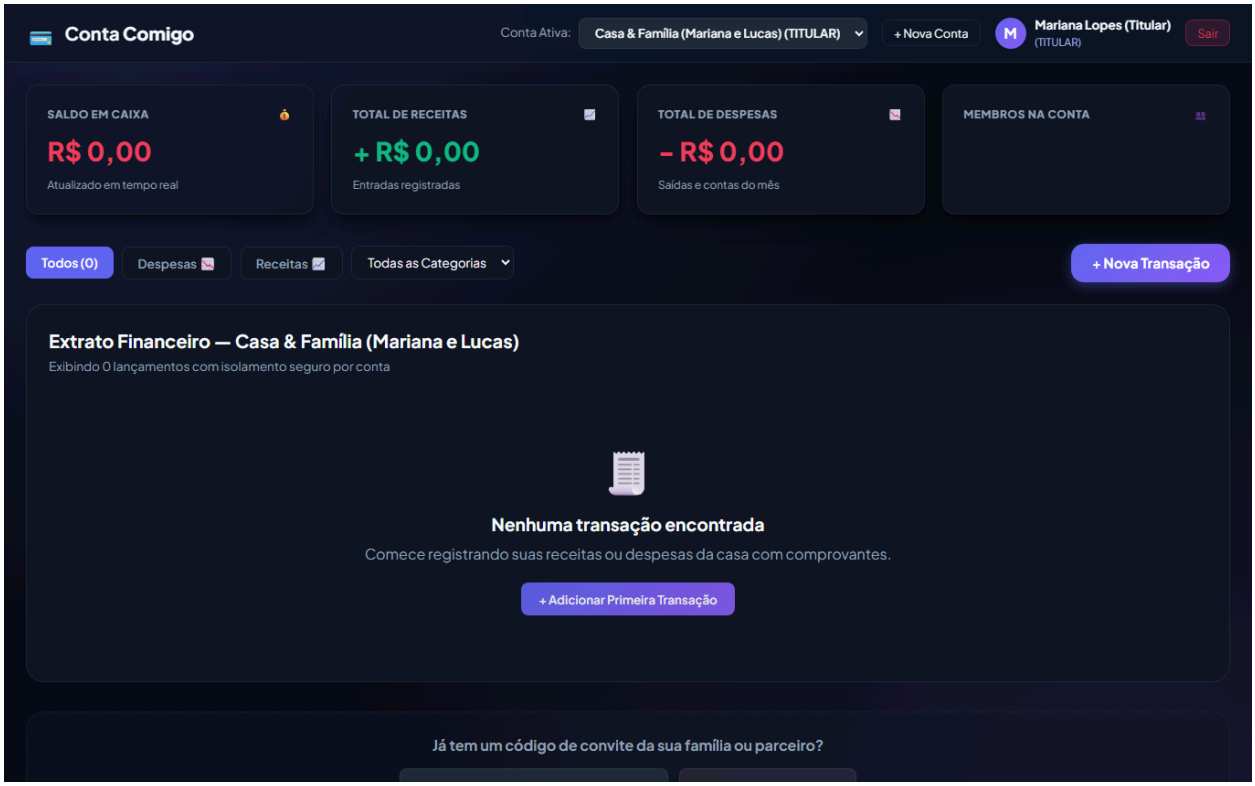


Figura 5: Atividade 10 — Dashboard do titular no sistema em produção (conta de demonstração sem lançamentos)

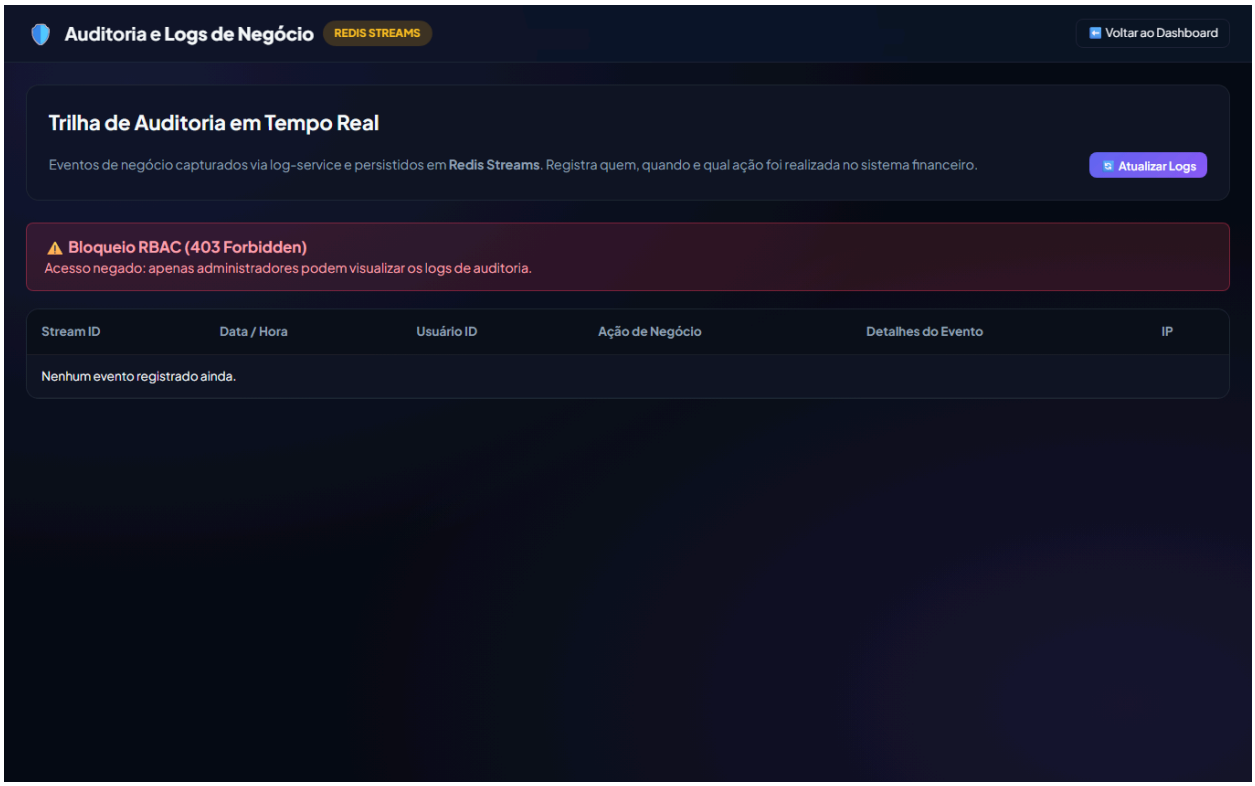


Figura 6: Atividade 10 — Bloqueio 403 Forbidden para papel sem permissão

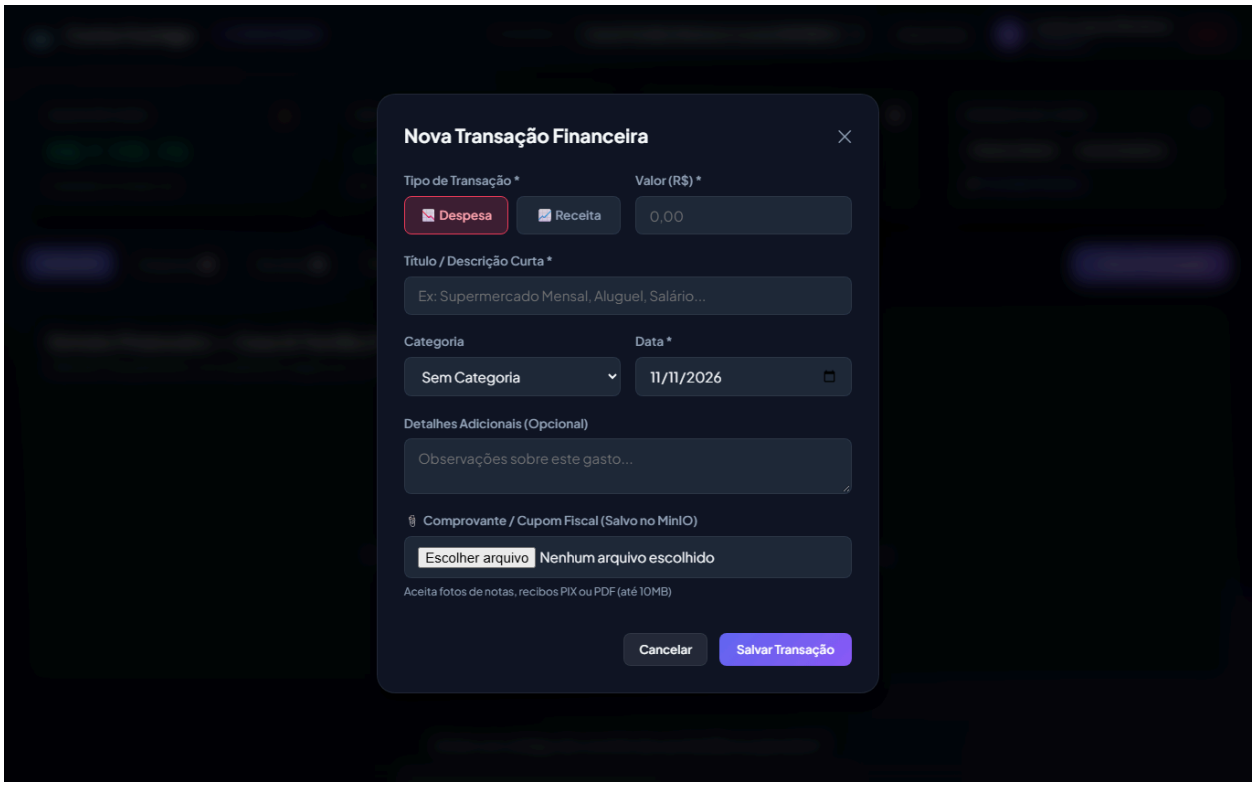


Figura 7: Atividade 10 — Modal de lançamento de transação com anexo de comprovante

Dificuldades e como foram resolvidas. Garantir que o isolamento de dados não dependesse de regras visuais do frontend, mas sim de validações profundas no backend. Para solucionar, criou-se a função auxiliar `checkUserAccountAccess` no backend, interceptando todas as rotas e validando a participação do usuário na conta solicitada antes de qualquer instrução SQL.

5.5 Atividade 11 — SaaS completo — planos, limites e assinatura com Stripe

Descrição	Planos recorrentes, checkout de assinatura, webhooks atualizando o plano, portal do cliente, limites por plano na API e painel admin.
Data planejada	13/11/2026
Data realizada	11/11/2026 10:53
Situação	entregue
Evidência	GitHub — commit dadf7a4 (ainda sem push) e prints do sistema; faltam os prints de checkout e webhook
Link	https://github.com/cleitongrodrigues/Controle_Financeiro/commit/dadf7a4

O que foi feito. Transformou-se a aplicação em um SaaS completo com receita recorrente: criação dos planos Free (gratuito com limite estrito de 3 transações mensais) e Pro (R$ 19,90/mês, uso ilimitado); checkout de assinatura no Stripe (`mode=subscription`); endpoint de webhooks com verificação criptográfica e tabela de idempotência `stripe_events` contra duplicações; suporte ao Customer Billing Portal para gerenciamento e cancelamento autônomo; validação rígida de limites na API (HTTP 403 ao tentar lançar a 4ª transação no Free); e criação do painel `/logs` exclusivo para administradores com exibição em tempo real do faturamento mensal recorrente (MRR).

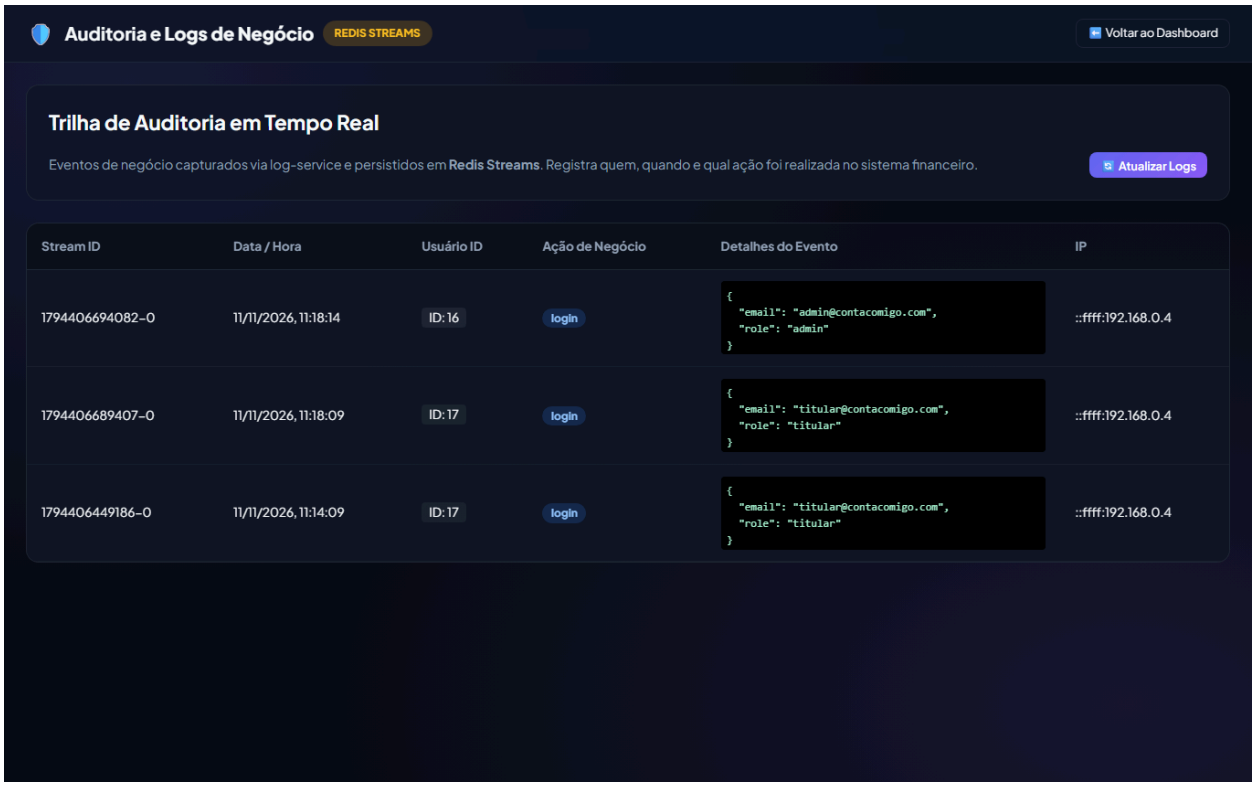


Figura 8: Atividade 11 — Painel de auditoria (Redis Streams) como está hoje no ar; os cards de MRR e usuários por plano ainda não aparecem porque a versão nova não foi publicada

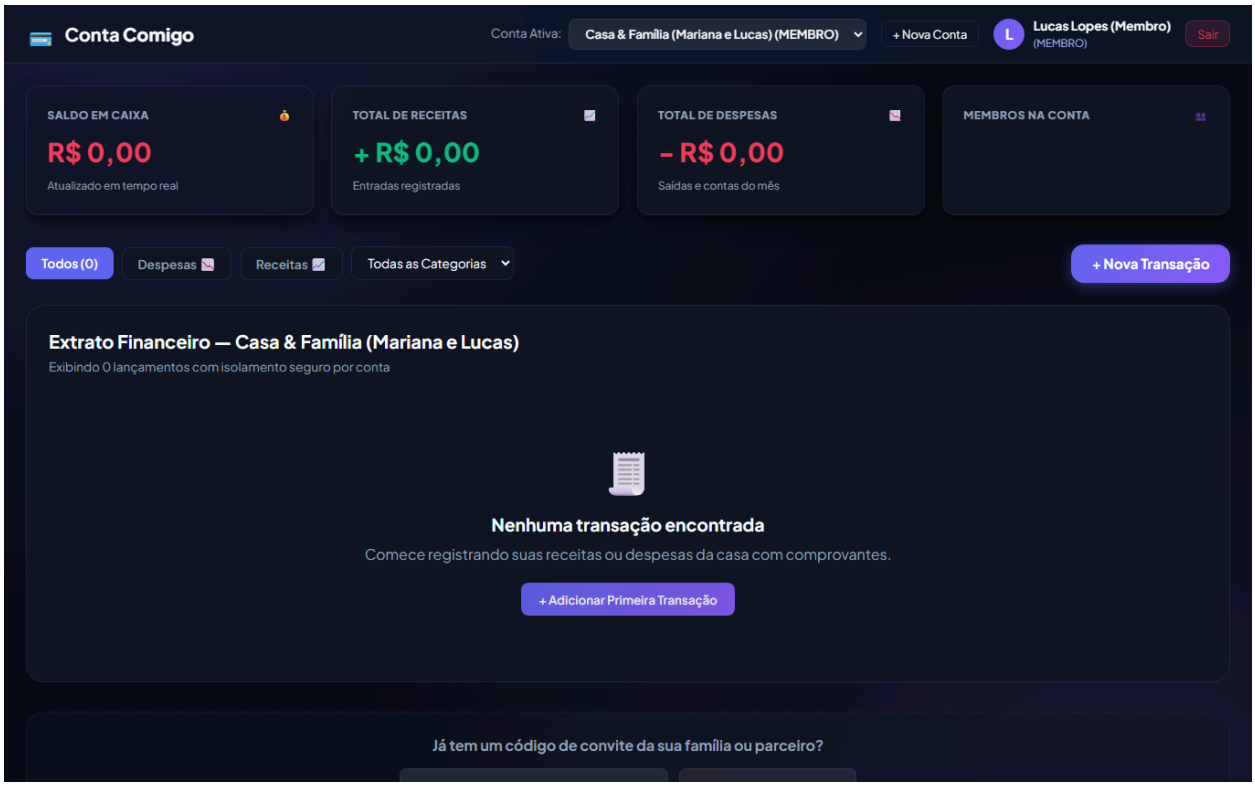


Figura 9: Atividade 11 — Dashboard do papel membro no sistema em produção

Dificuldades e como foram resolvidas. Tratar a concorrência e eventuais reenvios de notificações do Stripe sem gerar estados inconsistentes no banco. Isso foi superado com a criação da tabela `stripe_events`, que registra o identificador de cada webhook recebido sob chave primária, garantindo idempotência estrita a qualquer evento duplicado.

6 Considerações finais

A jornada prática percorrida no 2º bimestre de Computação em Nuvem consolidou competências essenciais que ultrapassam a codificação convencional. Compreendeu-se que a transição de um protótipo de software para um produto comercial viável exige um alinhamento rigoroso entre engenharia de microsserviços, integridade de dados, segurança em múltiplas camadas e modelos sustentáveis de monetização.

O aspecto mais desafiador do projeto residiu na orquestração assíncrona dos webhooks do Stripe associada à aplicação contínua de limites por plano na API. Garantir que a quota do plano Free seja checada a cada requisição de escrita no backend — e que o status da assinatura reflita a verdade do gateway externo sem depender do fluxo do navegador — evidenciou a importância dos princípios de resiliência e idempotência em arquiteturas distribuídas.

Para que o **Conta Comigo** evolua de um projeto acadêmico para uma startup em escala de produção comercial, os próximos passos consistem em implementar autenticação multifator (MFA), integração com a API do Open Finance (PIX automático e importação de extratos via Open Banking Brasil) e provisionamento da infraestrutura em nuvem pública utilizando práticas de Infraestrutura como Código (Terraform/Ansible) com esteira automatizada de CI/CD.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados.

Cleiton Rodrigues de Souza

Pompeia - SP, 13/11/2026